Department of Computer Science and Engineering

Course Code & Name	CSA0610 – Design and Analysis of Algorithms
Assessment	CO3 – Case Study – Divide-and-Conquer Engine of a Retail Data Analytics Platform
Course Outcome Assessed	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication.
Duration	1 Week
Mode	Individual
Student Reg. No.	192511308
Student Name	Divya Dharshini V S

Instructions: Analyse the case, apply divide-and-conquer concepts to each module, and justify every design decision with analysis. Show traces, recurrences and calculations.

CASE STUDY

Case Study – Divide-and-Conquer Engine of "KartIQ", a Retail Data Analytics Platform: KartIQ is the analytics platform of a national e-commerce marketplace that records about 10,000,000 orders a day and ranks 100,000 sellers. Customers, sellers and auditors complain that the platform is slow: the nightly order report misses its deadline, the live seller leaderboard freezes during sale events, order lookups take seconds, and product recommendations are refreshed only once a day. As the platform's algorithm engineer, you must analyse the four modules below and redesign them using divide-and-conquer algorithms.

Module 1 – Nightly Order Consolidation: All orders must be sorted by timestamp for the audit report. Orders arriving at the same second from different payment gateways must keep their original arrival order, because the auditors reconcile them gateway by gateway. Sample orders (timestamp, order ID): (105, O1), (102, O2), (105, O3), (101, O4), (102, O5), (103, O6).

Module 2 – Live Seller Leaderboard: Seller scores are re-sorted every few seconds. Between two refreshes only a few scores change, so the data is almost always nearly sorted. The current implementation is Quick Sort with the first element as the pivot, and the leaderboard freezes during every sale event. Sample scores: 42, 17, 63, 8, 55, 29.

Module 3 – Order Lookup: Customer-support staff look up an order by its timestamp in the consolidated list produced by Module 1. Several orders may share the same timestamp, and staff need the first of them.

Module 4 – Recommendation Scores: The score of every product for every customer segment is computed as $C = A \times B$, where A is the segment–feature preference matrix and B is the feature–product matrix; in production both are $n \times n$ with $n = 1{,}024$. Sample matrices: $A = [[1, 7], [3, 5]]$ and $B = [[1, 3], [7, 5]]$.

(a) Understand the Case: For each of the four modules, identify the computational requirement, the key issues (such as stability, memory, worst-case behaviour, the sorted-data precondition and matrix size) and the consequence of choosing a poor algorithm. Present your findings in a table.

(b) Module 1 – Merge Sort for Nightly Order Consolidation: Explain the divide, conquer and combine steps of Merge Sort and trace it on the sample orders, showing every merge. Demonstrate that orders with equal timestamps keep their gateway arrival order. Formulate and solve the recurrence, estimate the comparisons for 10,000,000 orders, and discuss the memory requirement and how it could be handled if the data does not fit in RAM.

(c) Module 2 – Quick Sort for the Live Leaderboard: Trace one partition step on the sample scores. Explain why the current first-element pivot makes the leaderboard slow, formulate the best-case and worst-case recurrences, and solve them. Estimate the comparisons for 100,000 sellers whose scores are already nearly in order. Propose and justify an improved pivot strategy.

(d) Module 3 – Binary Search for Order Lookup: Explain why the sorted order produced by Module 1 is essential, formulate and solve the recurrence for the number of comparisons, and compute the maximum comparisons for 10,000,000 orders. Modify the search so that it returns the first order with a given timestamp when several orders share it.

(e) Module 4 – Strassen's Method for Recommendation Scores: Write the seven Strassen products and the reconstruction formulas, and verify them on the 2×2 sample matrices. Formulate and solve the recurrence and compare the scalar multiplications with the conventional method for $n = 1{,}024$. Explain why Strassen's method may still not be faster in practice and propose a practical hybrid.

(f) Design and Justify the Final Solution: Present the final algorithmic design for the platform, one choice per module, with time complexity, extra memory and the reason for the choice. Identify the main trade-off of each decision and one condition under which you would change it.

Rubrics – Case Study

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25				
Application of Concepts	25				
Analysis	20				
Solution Design	20				
Clarity	10				
Total Marks					

Code of Conduct

I Divya Dharshini V S (192511308) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: 01/11/2026

DESIGN AND ANALYSIS OF ALGORITHMS

CO3 – CASE STUDY

DIVIDE-AND-CONQUER ENGINE OF KARTIQ RETAIL DATA ANALYTICS PLATFORM

(a) UNDERSTAND THE CASE

1. Introduction

KartIQ is a retail data analytics platform that processes millions of orders and seller records every day. Its existing algorithms cause delays in order sorting, seller ranking, order searching and recommendation generation.

Divide and Conquer is an algorithm design technique that divides a large problem into smaller subproblems, solves them independently and combines their results. The four modules require different algorithms based on their data characteristics and computational requirements.

2. Analysis of the Four Modules

Module	Computational Requirement	Key Issues	Suitable Algorithm
Module 1 – Order Consolidation	Sort 10 million orders by timestamp	Stability and large memory requirement	Merge Sort
Module 2 – Seller Leaderboard	Frequently sort 100,000 seller scores	Nearly sorted data and Quick Sort worst case	Improved Quick Sort
Module 3 – Order Lookup	Find the first order with a timestamp	Sorted data and duplicate timestamps	Modified Binary Search
Module 4 – Recommendations	Multiply two 1024 × 1024 matrices	Large number of scalar multiplications	Strassen's Method

3. Issues and Consequences of Poor Algorithm Selection

Module 1 – Nightly Order Consolidation

Orders having identical timestamps must retain their original arrival order. Therefore, stability is essential. An unstable sorting algorithm may rearrange orders from different payment gateways, causing audit errors.

Merge Sort is suitable because it guarantees O(n log n) time and preserves the relative order of equal elements when implemented correctly.

Module 2 – Live Seller Leaderboard

Seller scores change only slightly between refreshes, making the data nearly sorted. Choosing the first element as the Quick Sort pivot repeatedly creates unbalanced partitions.

This may increase execution time to $O(n^2)$, freezing the leaderboard during sale events.

Module 3 – Order Lookup

The consolidated order list is already sorted by timestamp. Binary Search can use this property to eliminate half of the remaining search space in every iteration.

Ordinary Binary Search may return any matching timestamp. Therefore, a modified version is required to locate the first occurrence.

Module 4 – Recommendation Scores

Matrix multiplication requires a large number of scalar multiplications. Conventional multiplication takes $O(n^3)$ time.

Strassen's algorithm reduces the number of recursive multiplications from eight to seven, improving the asymptotic complexity to O(n^2.807).

4. Overall Observation

The algorithms must be selected according to the nature of each module rather than using one algorithm everywhere. A poor selection can increase processing time, consume unnecessary memory, produce incorrect audit ordering or delay customer recommendations. Therefore, KartIQ requires stable sorting, balanced partitioning, efficient searching and optimized matrix multiplication.

(b) MODULE 1 – MERGE SORT FOR NIGHTLY ORDER CONSOLIDATION

1. Introduction

Merge Sort is a stable sorting algorithm based on the Divide-and-Conquer technique. It sorts elements by repeatedly dividing the input into smaller halves, sorting them and merging the sorted halves.

KartIQ processes approximately 10,000,000 orders daily. The orders must be arranged according to timestamps while maintaining the original arrival order of orders having equal timestamps.

Objective: To sort all orders by timestamp without changing the relative order of equal-timestamp orders.

2. Working Principle

Merge Sort follows three important steps:

1. Divide:

- Divide the given array into two approximately equal halves.
- Continue dividing until each subarray contains one element.

2. Conquer:

- A single-element array is already sorted.
- Recursively sort the smaller subarrays.

3. Combine:

- Compare the first elements of both sorted subarrays.
- Copy the smaller element into the result.
- If timestamps are equal, select the element from the left subarray first.
- Continue until all elements are merged.

Selecting the left element when timestamps are equal ensures stability.

3. Sample Input

Orders:

Original Position	Timestamp	Order ID
1	105	O1
2	102	O2
3	105	O3
4	101	O4
5	102	O5
6	103	O6

4. Divide-and-Conquer Trace

Initial Array:

[(105,O1), (102,O2), (105,O3), (101,O4), (102,O5), (103,O6)]

Step 1 – Divide

Left half:

[(105,O1), (102,O2), (105,O3)]

Right half:

[(101,O4), (102,O5), (103,O6)]

Step 2 – Divide Further

Left:

[(105,O1)] and [(102,O2), (105,O3)]

Right:

[(101,O4)] and [(102,O5), (103,O6)]

Continue dividing until individual elements are obtained.

Step 3 – Merge Every Subarray

Merge	Comparison	Result
1	102 < 105	[(102,O2), (105,O3)]
2	102 < 105	[(102,O2), (105,O1), (105,O3)]
3	102 < 103	[(102,O5), (103,O6)]
4	101 < 102	[(101,O4), (102,O5), (103,O6)]
5	Merge both sorted halves	Final sorted array

Final Sorted Orders:

[(101,O4), (102,O2), (102,O5), (103,O6), (105,O1), (105,O3)]

5. Stability Verification

The original positions of equal timestamps are:

- Timestamp 102: O2 appeared before O5.
- Timestamp 105: O1 appeared before O3.

The final sorted order also maintains:

O2 before O5 and O1 before O3.

Therefore, Merge Sort preserves the original arrival order of equal-timestamp orders.

This property is essential for accurate gateway-wise auditing.

6. Algorithm

1) START.
2) INPUT the order array.
3) If the array contains one element, return it.
4) Divide the array into two halves.
5) Recursively sort both halves.
6) Compare timestamps and merge the halves.
7) If timestamps are equal, select from the left half.

8) OUTPUT the sorted orders.

9) STOP.

7. Pseudocode

```
MERGE_SORT(A, l, r)
BEGIN
   IF l < r
      m = (l+r)/2
      MERGE_SORT(A,l,m)
      MERGE_SORT(A,m+1,r)
      MERGE(A,l,m,r)
   ENDIF
END
MERGE(A,l,m,r)
BEGIN
   Merge both halves in ascending order.
   If timestamps are equal, take from left.
   Copy remaining elements.
END
```

8. Recurrence Relation

Let T(n) represent the execution time for sorting n elements. Merge Sort divides the problem into two halves. Each half requires T(n/2) time, and merging requires O(n) time. Therefore,

$T(n) = 2T(n/2) + cn$

where c is the constant cost per element during merging.

For n = 1,

$T(1) = c_0$

Using the Master Theorem:

$a = 2, b = 2, f(n) = cn$

$n^{(\log_2 2)} = n$

Since $f(n) = \Theta(n)$,

$T(n) = \Theta(n \log_2 n)$

Therefore, the time complexity of Merge Sort is O(n log n).

9. Mathematical Comparison Analysis

For $n = 2^k$, the maximum number of comparisons satisfies:

$C(n) = 2C(n/2) + (n-1)$

Expanding:

$C(n) = 2[2C(n/4) + (n/2 - 1)] + (n-1)$

$C(n) = 4C(n/4) + 2n - 3$

After k levels:

$C(n) = nC(1) + n \log_2 n - n + 1$

Taking $C(1) = 0$:

$C(n) = n \log_2 n - n + 1$

For 10,000,000 orders:

$\log_2(10{,}000{,}000) \approx 23.2535$

Approximate comparison count:

$C(n) \approx n \log_2 n$

$= 10{,}000{,}000 \times 23.2535$

$\approx 232{,}534{,}967$ comparisons.

Thus, approximately 232.5 million comparisons are required at this scale.

10. Space Complexity

Merge Sort requires an auxiliary array to temporarily store merged elements.

- Input storage: O(n).
- Temporary merge array: O(n).
- Recursion stack: O(log n).

Therefore:

Auxiliary Space = O(n) + O(log n)

Overall auxiliary space complexity = O(n).

11. Handling Data That Does Not Fit in RAM

Storing 10 million orders may require substantial memory. If the entire dataset does not fit into RAM, KartIQ can use External Merge Sort.

Working:

1. Divide the order file into smaller chunks.
2. Load one chunk into RAM.
3. Sort each chunk using Merge Sort.
4. Store each sorted chunk on secondary storage.
5. Merge the sorted files using buffers.
6. Generate the final sorted order file.

This approach reduces RAM requirements and supports very large datasets.

12. Conclusion

Merge Sort is the best choice for KartIQ's nightly order consolidation because it guarantees O(n log n) time and preserves the relative order of equal timestamps.

External Merge Sort can handle datasets larger than available RAM, ensuring reliable and efficient audit reporting.

(c) MODULE 2 – QUICK SORT FOR THE LIVE LEADERBOARD

1. Introduction

Quick Sort is a Divide-and-Conquer sorting algorithm that selects a pivot and partitions the array into smaller and larger elements.

The left partition contains elements smaller than the pivot, while the right partition contains elements greater than or equal to the pivot, according to the selected partition rule.

KartIQ uses Quick Sort to update the leaderboard of 100,000 sellers.

However, selecting the first element as the pivot creates poor performance when the data is already nearly sorted.

2. Trace of One Partition Step

Sample Scores:

[42, 17, 63, 8, 55, 29]

Choose the first element as pivot.

Pivot = 42

Using a partition method that moves elements smaller than the pivot to its left:

Step	Operation	Array
1	Select pivot 42	42, 17, 63, 8, 55, 29
2	17 < 42	42, 17, 63, 8, 55, 29
3	63 > 42	No change
4	8 < 42, swap with 63	42, 17, 8, 63, 55, 29
5	55 > 42	No change
6	29 < 42, swap with 63	42, 17, 8, 29, 55, 63
7	Place pivot in its correct position	29, 17, 8, 42, 55, 63

Result:

[29, 17, 8] | 42 | [55, 63]

The pivot 42 is now in its correct sorted position.

3. Why the Current Implementation Becomes Slow

The current implementation always selects the first element as the pivot.

When the scores are already sorted:

[8, 17, 29, 42, 55, 63]

The first element becomes the smallest pivot.

The partitions become:

Left partition = 0 elements.

Right partition = n-1 elements.

The same imbalance occurs repeatedly.

Consequently, the algorithm performs unnecessary comparisons and approaches quadratic time complexity. For 100,000 nearly sorted sellers, this can significantly delay leaderboard updates.

4. Best-Case Recurrence

In the best case, the pivot divides the array into two equal halves.

$T(n) = 2T(n/2) + cn$

Using the Master Theorem:

$a = 2, b = 2, f(n) = cn$

Therefore:

$T(n) = \Theta(n \log n)$

The algorithm performs approximately:

$C(n) = n \log_2 n - n + 1$

For 100,000 sellers:

$\log_2(100,000) \approx 16.6096$

$C(n) \approx 100,000 \times 16.6096 - 100,000 + 1$

$C(n) \approx 1,560,965$ comparisons.

Thus, balanced partitioning provides efficient performance.

5. Worst-Case Recurrence

In the worst case, one partition contains n-1 elements and the other contains zero elements.

$T(n) = T(n-1) + cn$

Expanding:

$T(n) = T(n-2) + 2c + cn$

$T(n) = T(n-3) + 3c + cn + c(n-1)$

Continuing:

$T(n) = T(1) + c[1 + 2 + 3 + ... + (n-1)]$

Using the arithmetic series formula:

$T(n) = c \times n(n-1)/2$

Therefore:

$T(n) = \Theta(n^2)$

Maximum comparisons for 100,000 sellers:

$C(n) = n(n-1)/2$

$= 100,000 \times 99,999 / 2$

= 4,999,950,000 comparisons.

This explains why the leaderboard can freeze during sale events.

6. Improved Pivot Strategy

KartIQ should use Median-of-Three Pivot Selection.

It selects three elements:

- First element.
- Middle element.
- Last element.

The median of these three values is selected as the pivot.

For example, consider:

[8, 17, 29, 42, 55, 63]

First = 8

Middle = 29

Last = 63

Median = 29

Instead of selecting the smallest element, the algorithm selects 29, producing more balanced partitions.

For frequently changing data, randomized pivot selection is another useful option.

7. Improved Algorithm

1) START.
2) INPUT the seller scores.
3) Select first, middle and last elements.
4) Choose their median as pivot.
5) Partition the array around the pivot.
6) Recursively sort the left and right partitions.
7) OUTPUT the sorted scores.
8) STOP.

8. Pseudocode

```
QUICK_SORT(A,l,r)
BEGIN
  IF l < r
    p = MEDIAN_OF_THREE(A,l,r)
    PARTITION(A,l,r,p)
    QUICK_SORT(A,l,p-1)
```

```
    QUICK_SORT(A,p+1,r)
  ENDIF
END
```

9. Complexity Comparison

Case	Partition	Time Complexity
Best	Equal halves	O(n log n)
Average	Reasonably balanced	O(n log n)
Worst	0 and n-1 elements	$O(n^2)$

The median-of-three strategy reduces the likelihood of repeatedly selecting extreme pivots. However, it does not eliminate the theoretical $O(n^2)$ worst case.

10. Space Complexity

Quick Sort performs partitioning in the original array.

- Partitioning space: O(1).
- Recursion stack in balanced case: O(log n).
- Recursion stack in worst case: O(n).

Therefore:

Best/Average Auxiliary Space = O(log n)

Worst Auxiliary Space = O(n)

11. Conclusion

Quick Sort is suitable for the live leaderboard because it provides efficient average-case performance and in-place partitioning.

However, the first-element pivot is unsuitable for nearly sorted data.

Median-of-three pivot selection improves partition balance and reduces the likelihood of quadratic execution time, helping KartIQ maintain responsive leaderboard updates.

(d) MODULE 3 – BINARY SEARCH FOR ORDER LOOKUP

1. Introduction

Binary Search is an efficient searching algorithm based on the Divide-and-Conquer technique. It repeatedly divides the sorted search space into two halves and eliminates the half that cannot contain the required element. KartIQ uses Binary Search to locate orders by timestamp in the consolidated order list.

2. Importance of Sorted Data

Binary Search requires the input array to be sorted. Module 1 uses stable Merge Sort to arrange orders in ascending timestamp order. Example: [101, 102, 102, 103, 105, 105]

If the array is not sorted, Binary Search cannot correctly eliminate half of the elements.

Therefore, the sorted output from Module 1 is essential for Module 3.

3. Working Principle

1) Set low = 0 and high = n-1.
2) Calculate the middle position.
3) Compare the middle timestamp with the target.
4) If the middle value is smaller, search the right half.
5) If it is greater, search the left half.
6) If it is equal, record the position and continue searching the left half.
7) Return the first matching position.

4. Sample Example

Sorted timestamps: [101, 102, 102, 103, 105, 105]

Target timestamp = 102

Iteration	Low	High	Mid	Value	Action
1	0	5	2	102	Record position, search left
2	0	1	0	101	Search right
3	1	1	1	102	Record first position

Final Result:

First occurrence = Index 1.

Order ID = O2.

The algorithm correctly returns O2 instead of O5.

5. Recurrence Relation

Let T(n) represent the number of comparisons.

Therefore:

$T(n) = T(n/2) + c$

where c represents a constant number of comparisons per iteration.

Base condition:

$T(1) = 1$

Expanding:

$T(n) = T(n/2) + c$

$T(n) = T(n/4) + 2c$

$T(n) = T(n/8) + 3c$

After k iterations:

$T(n) = T(n/2^k) + kc$

The search terminates when:

$n/2^k = 1$

Therefore: $2^k = n$

$k = \log_2 n$

Hence:

$T(n) = \Theta(\log n)$

Time Complexity = O(log n).

6. Maximum Comparisons for 10,000,000 Orders

Number of orders:

$n = 10,000,000$

Maximum comparisons for ordinary binary search:

$C(n) \leq \text{floor}(\log_2 n) + 1$

Since:

$2^{23} = 8,388,608$

$2^{24} = 16,777,216$

Therefore:

$\text{floor}(\log_2 10,000,000) + 1 = 23 + 1$

Maximum comparisons = 24.

7. Modified Algorithm – First Occurrence

The modification is to continue searching the left half whenever a match is found. Instead of immediately returning the matching position, store it in a variable called result. This ensures that an earlier duplicate is not missed.

8. Pseudocode

```
FIRST_SEARCH(A,n,key)
BEGIN
   low = 0, high = n-1, result = -1
   WHILE low <= high
      mid = (low+high)/2
      IF A[mid] == key
         result = mid
         high = mid-1
      ELSE IF A[mid] < key
```

```
            low = mid+1
        ELSE
            high = mid-1
    RETURN result
END
```

9. Step-by-Step Explanation

- Initially, the complete sorted array is considered.
- The middle timestamp is checked against the target.
- If the target is found, its position is stored.
- The search continues towards the left side.
- If another matching timestamp exists earlier, the result is updated.
- When the search ends, the earliest matching position is returned.
- If no matching timestamp exists, -1 is returned.

10. Space Complexity

The iterative algorithm uses only three additional variables:

- low
- high
- mid and result

The number of additional variables remains constant irrespective of input size.

Therefore: Auxiliary Space = O(1).

11. Applications

- Customer order lookup.
- Finding transaction timestamps.
- Searching sorted product records.

12. Conclusion

Modified Binary Search is suitable for KartIQ because it searches 10 million sorted orders using at most 24 loop iterations. It also identifies the first occurrence of duplicate timestamps, ensuring accurate customer-support and audit operations.

(e) MODULE 4 – STRASSEN'S METHOD FOR RECOMMENDATION SCORES

1. Introduction

Matrix multiplication is an important operation in recommendation systems.

KartIQ calculates product recommendation scores using:

$C = A \times B$

Here:

- A represents the segment-feature preference matrix.
- B represents the feature-product matrix.
- C represents the final segment-product recommendation scores.

For production, both matrices are 1024 × 1024.

Conventional matrix multiplication requires $O(n^3)$ time. Strassen's algorithm reduces the number of recursive scalar multiplications from eight to seven.

2. Working Principle

Strassen's method divides both matrices into four equal-sized submatrices.

Let:

$$A = \begin{vmatrix} a & b \\ c & d \end{vmatrix}$$

$$B = \begin{vmatrix} e & f \\ g & h \end{vmatrix}$$

Instead of performing eight recursive matrix multiplications, it calculates seven products.

3. Seven Strassen Products

M1 = (a+d)(e+h)

M2 = (c+d)e

M3 = a(f-h)

M4 = d(g-e)

M5 = (a+b)h

M6 = (c-a)(e+f)

M7 = (b-d)(g+h)

These seven products are combined to calculate the four elements of the result matrix.

4. Reconstruction Formulas

C11 = M1 + M4 - M5 + M7

C12 = M3 + M5

C21 = M2 + M4

C22 = M1 - M2 + M3 + M6

Therefore:

C = | C11 C12 |
| C21 C22 |

5. Verification Using the Sample Matrices

Given:

A = | 1 7 |
| 3 5 |

B = | 1 3 |
| 7 5 |

Here:

a=1, b=7, c=3, d=5, e=1, f=3, g=7, h=5

Calculate Seven Products

M1 = (1+5)(1+5)

M1 = 6 × 6 = 36

M2 = (3+5) × 1

M2 = 8

M3 = 1 × (3-5)

M3 = -2

M4 = 5 × (7-1)

M4 = 30

M5 = (1+7) × 5

M5 = 40

M6 = (3-1)(1+3)

M6 = 2 × 4 = 8

M7 = (7-5)(7+5)

M7 = 2 × 12 = 24

6. Calculate the Result Matrix

C11 = M1 + M4 - M5 + M7

= 36 + 30 - 40 + 24

= 50

C12 = M3 + M5

= -2 + 40

= 38

C21 = M2 + M4

= 8 + 30

= 38

C22 = M1 - M2 + M3 + M6

= 36 - 8 - 2 + 8

= 34

Final Result:

C = | 50 38 |
| 38 34 |

7. Verification Using Conventional Multiplication

For the first element:

C11 = (1×1) + (7×7)

= 1 + 49 = 50

For the second element:

C12 = (1×3) + (7×5)

= 3 + 35 = 38

For the third element:

C21 = (3×1) + (5×7)

= 3 + 35 = 38

For the fourth element:

C22 = (3×3) + (5×5)

= 9 + 25 = 34

Therefore, conventional multiplication also produces:

C = | 50 38 |
| 38 34 |

Both methods produce the same result, verifying Strassen's formulas.

8. Recurrence Relation

Strassen's algorithm divides each n × n matrix into four submatrices of size n/2.

It performs seven recursive multiplications and $O(n^2)$ additions and subtractions.

Therefore: $T(n) = 7T(n/2) + cn^2$

Base condition:

$T(1) = c_0$

Using the Master Theorem:

$a = 7, b = 2$

$f(n) = cn^2$

Calculate:

$n^{(\log_2 7)} \approx n^{2.8074}$

Since:

$n^2 = O(n^{2.8074})$

The first case of the Master Theorem applies.

Therefore:

$T(n) = \Theta(n^{\log_2 7})$

$T(n) = O(n^{2.8074})$

9. Scalar Multiplication Comparison for n = 1024

Given:

$n = 1024 = 2^{10}$

Conventional Matrix Multiplication

Number of scalar multiplications:

$n^3 = 1024^3$

= 1,073,741,824

Strassen's Matrix Multiplication

Number of scalar multiplications:

$7^{\log_2 n}$

$= 7^{10} = 282,475,249$

10. Comparison Table

Parameter	Conventional Method	Strassen's Method
Recursive multiplications	8	7
Time complexity	$O(n^3)$	O(n^2.8074)
Multiplications for 1024 × 1024	1,073,741,824	282,475,249
Addition overhead	Lower	Higher
Practical implementation	Simpler	More complex

Reduction in scalar multiplications:

1,073,741,824 - 282,475,249 = 791,266,575

Therefore, Strassen's method reduces the number of scalar multiplications by approximately 73.7%.

11. Why Strassen May Not Be Faster in Practice

Strassen's algorithm has better asymptotic complexity, it may not always perform faster.

Reasons:

1. It requires additional matrix additions and subtractions.
2. Temporary matrices increase memory usage.
3. Recursive function calls introduce overhead.
4. Small matrices may be processed faster using conventional multiplication.
5. Memory allocation and data movement can reduce performance.
6. Floating-point rounding errors may increase in numerical applications.

Therefore, theoretical improvement does not always guarantee practical improvement.

12. Practical Hybrid Approach

KartIQ can implement a hybrid matrix multiplication strategy.

Working:

1. Divide large matrices using Strassen's method.
2. Continue recursive division until a selected threshold is reached.
3. Use conventional matrix multiplication for smaller submatrices.
4. Combine the results to generate the final recommendation matrix.

For example, KartIQ may use Strassen's method for larger matrices and switch to conventional multiplication when the submatrix size becomes 64 × 64 or smaller.

13. Space Complexity

Strassen's method requires temporary storage for intermediate matrices. A straightforward implementation may require $O(n^2)$ auxiliary memory at a time, depending on how temporary matrices are allocated and reused. Careful memory management and in-place workspace reuse can reduce unnecessary allocations.

14. Conclusion

Strassen's method is suitable for KartIQ's large recommendation matrices because it reduces the multiplication count from eight to seven at every recursive level. For 1024 × 1024 matrices, it requires approximately 282 million scalar multiplications instead of 1.07 billion. A hybrid approach combining Strassen's and conventional multiplication provides a practical balance between speed and memory usage.

(f) DESIGN AND JUSTIFY THE FINAL SOLUTION

1. Introduction

KartIQ requires an efficient algorithmic architecture to process millions of orders, update seller rankings, search customer transactions and calculate product recommendation scores. The four modules use different Divide-and-Conquer algorithms based on their individual requirements. The final design must balance execution time, memory usage, stability and practical performance.

2. Final Algorithmic Design

Module	Selected Algorithm	Time Complexity	Extra Memory	Reason
Order Consolidation	Stable Merge Sort	O(n log n)	O(n)	Guarantees stable sorting
Seller Leaderboard	Median-of-Three Quick Sort	Expected O(n log n), worst $O(n^2)$	O(log n) expected stack	Reduces unbalanced partitions
Order Lookup	Modified Binary Search	O(log n)	O(1)	Finds the first matching timestamp
Recommendation Scores	Hybrid Strassen's Method	O(n^2.8074) for large matrices	$O(n^2)$ workspace	Reduces recursive multiplications

3. Module-Wise Design Justification

Module 1 – Stable Merge Sort

Merge Sort is selected because KartIQ must sort 10 million orders while preserving the original arrival order of identical timestamps. Its guaranteed O(n log n) performance makes it suitable for nightly order consolidation.

Main trade-off:

It requires O(n) additional memory.

Condition for changing the decision:

If the complete dataset cannot fit into RAM, External Merge Sort should be used to process sorted chunks from secondary storage.

Module 2 – Improved Quick Sort

Median-of-three pivot selection is used to improve partition balance. It reduces the likelihood of worst-case behaviour when seller scores are already nearly sorted. Its expected time complexity is O(n log n), with $O(n^2)$ worst-case time.

Main trade-off:

Quick Sort does not guarantee O(n log n) in every case.

Condition for changing the decision:

If guaranteed worst-case O(n log n) performance is required, an Introsort-based implementation can be used. It switches to Heap Sort when recursion becomes too deep.

Module 3 – Modified Binary Search

Binary Search is selected because Module 1 already produces a sorted timestamp list. The modified algorithm records a match and continues searching the left half to identify the first occurrence.

Its time complexity is O(log n), and iterative implementation requires O(1) auxiliary space.

Main trade-off:

It requires sorted input.

Condition for changing the decision:

If the order records are frequently updated and cannot be maintained in sorted order, an appropriate indexed data structure such as a balanced search tree may be considered.

Module 4 – Hybrid Strassen's Method

Strassen's method is selected because it reduces the recursive scalar multiplication count from eight to seven.

Its time complexity is O(n^2.8074), making it attractive for large recommendation matrices.

Main trade-off:

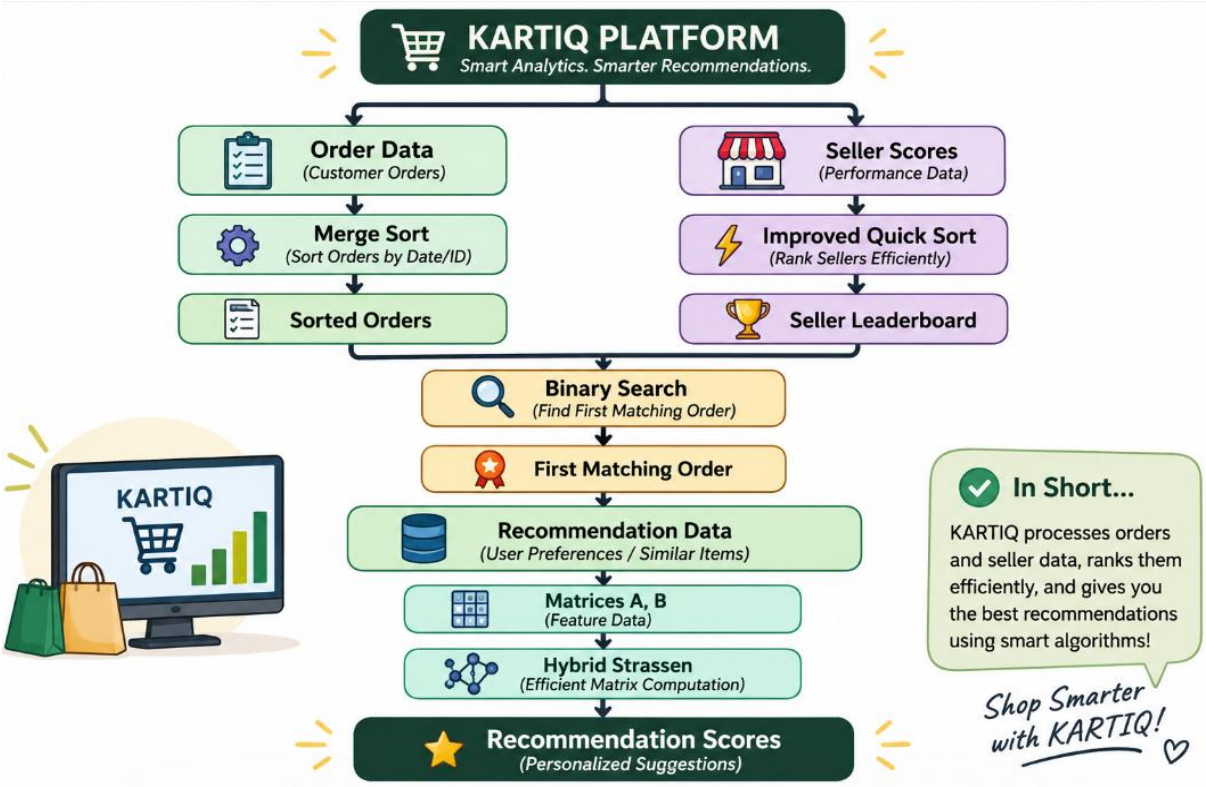
It introduces additional additions, temporary storage and recursion overhead.

Condition for changing the decision:

For small matrices or workloads where Strassen's overhead exceeds its benefits, conventional matrix multiplication should be used.

4. Final Platform Workflow

The complete KartIQ architecture follows this sequence:



5. Overall Complexity Analysis

Let: n = number of orders, s = number of sellers, m = matrix dimension.

Order Consolidation:

$T(n) = O(n \log n)$

Seller Leaderboard:

Expected $T(s) = O(s \log s)$, Worst $T(s) = O(s^2)$

Order Lookup:

$T(n) = O(\log n)$

Recommendation Calculation:

$T(m) = O(m^{2.8074})$

These complexities show that each module uses an algorithm suitable for its input characteristics.

6. Overall Advantages

- Reduces the time required for nightly order consolidation.
- Maintains stable ordering for audit accuracy.
- Improves leaderboard responsiveness during sale events.
- Provides fast customer order lookup.
- Reduces scalar multiplications in recommendation calculations.

7. Conclusion

The proposed KartIQ architecture combines Stable Merge Sort, improved Quick Sort, Modified Binary Search and Hybrid Strassen's Method. Each algorithm is selected according to the specific requirements of its module. The design improves sorting, searching and recommendation processing while considering stability, memory usage and worst-case behaviour. Therefore, the proposed Divide-and-Conquer architecture provides an efficient and scalable solution for KartIQ's large-scale retail analytics platform.

Question	Topic	What It Asks
A	Problem Analysis	Identify the requirements, challenges and suitable algorithms for KartIQ.
B	Merge Sort	Explain stable sorting with a step-by-step trace, complexity and memory analysis.
C	Quick Sort	Trace partitioning, analyse worst-case performance and suggest an improved pivot.
D	Binary Search	Find the first occurrence of a duplicate timestamp and analyse its complexity.
E	Strassen’s Algorithm	Perform matrix multiplication, verify the result and compare its efficiency with conventional multiplication.
F	Final System Design	Select the best algorithm for each module and justify the choices and trade-offs.